

Actividad Integradora Reporte Equipo 5

Análisis y diseño de algoritmos avanzados

Grupo 602

Carlos Delgado Contreras A01712819

Andrea Iliana Cantú Mayorga A01753419

Jesus Rodriguez Mendoza A01713550



**Tecnológico
de Monterrey**

Introducción

Cuando un dispositivo envía información a otro, lo hace como una serie de bits organizados en cabecera, datos y cola. Una persona malintencionada puede interceptar la transmisión, modificar alguna de sus partes e insertar scripts que le den control del dispositivo receptor. En la industria estas transmisiones contienen millones de caracteres, por lo que se necesitan algoritmos eficientes en tiempo y memoria.

Este reporte busca responder tres preguntas: si conocemos secuencias de código malicioso, ¿podemos identificarlas dentro de una transmisión y saber dónde comienzan? ¿Podemos detectar estructuras regulares como los palíndromos? Y si dos transmisiones fueron intervenidas con el mismo código, ¿podemos proponer cuál es a partir de lo que tienen en común?

Para ello se implementaron tres algoritmos:

Búsqueda de código malicioso: Rabin-Karp con Polynomial Rolling Hash, que reduce el costo promedio de $O(n \cdot m)$ a $O(n+m)$.

Palíndromo más largo: algoritmo de Manacher, que lo resuelve en tiempo lineal.

Subcadena común más larga: programación dinámica, que identifica el bloque compartido entre dos transmisiones como candidato a código malicioso.

Parte 1: Búsqueda de código malicioso

1.1 Contexto de Problemática

Esta sección cubre la solución propuesta para el siguiente reto: Dado cierta selección de transmisiones de información con datos alfanuméricos y dado un conjunto de patrones reconocidos como código malicioso. Se busca reconocer dichos patrones maliciosos en el contenido de las transmisiones. Indicando la posición en donde se encuentra el inicio de este patrón en la transmisión en caso de identificarlo. En la industria, estas transmisiones generalmente contienen millones de datos. Se busca que la identificación de patrones maliciosos en el contenido de las transmisiones sea eficiente tanto en tiempo como en complejidad.

1.2 Solución Propuesta

Existen múltiples técnicas de búsqueda de patrones. Para esta problemática propusimos un algoritmo de Rabin-Karp de patrón único. Dicho algoritmo es altamente eficiente para la búsqueda de múltiples patrones simultáneamente. Logrando mantener una alta eficiencia.

El algoritmo de Rabin-Karp propone que, en lugar de comprobar si un patrón encaja en cada posición del texto mediante fuerza bruta ($O(n \cdot m)$), calcular el valor hash del patrón y el de subcadenas del mismo tamaño del texto incrementa considerablemente la eficiencia temporal $O(n + m)$. El principio fundamental detrás del algoritmo Rabin-Karp es la técnica matemática Polynomial Rolling Hash. Esta técnica, también llamada Hash Rodante, permite actualizar el valor del Hash de la ventana actual con el valor de la siguiente subcadena del texto original de manera rápida y super eficiente. En lugar de tener que recalcularse el valor

hash entero al avanzar un carácter en el texto, el Polynomial Rolling Hash utiliza una técnica matemática para poder realizarlo a velocidad $O(1)$. Una vez Polynomial Rolling Hash encuentra una coincidencia de valores Hash, Rabin-Karp incluye una comparación **carácter por carácter** de la subcadena contra el patrón. Esto para descartar falsos positivos debido a colisiones en los valores hash.

Para convertir subcadenas en un entero (valor hash), el Polynomial Rolling Hash evalúa las subcadenas como si fueran un **polinomio** donde cada letra es un coeficiente. Por ejemplo para una subcadena/ventana de 4 caracteres, el cálculo se vería de la siguiente manera:

$$\text{Hash} = c_0 \cdot p^3 + c_1 \cdot p^2 + c_2 \cdot p^1 + c_3 \cdot p^0$$

Esta estructura genera un patrón en donde el primer elemento (primero a la izquierda) siempre tiene potencia de p^{m-1} donde M es el tamaño de la subcadena. Cuando queremos mover un elemento a la derecha, debemos de restarle al valor total del hash el valor el primer elemento & sumarle el valor del nuevo elemento. Con la siguiente secuencia de pasos logramos mover la ventana de subcadena a velocidad $O(1)$.

A. Restar valor de salida

Eliminar el primer elemento del valor hash quiere decir que debemos de restar el valor del coeficiente por su potencia ($c_0 \cdot p^{m-1}$) y multiplicarlo por el módulo para mantener los valores en rangos manejables. En programación, los modelos tienen la peculiaridad de poder regresar valores negativos cuando matemáticamente no deberían. Aquí se aplica un truco que consiste en sumar el módulo. Esta operación no altera el valor en sí sino que lo mantiene en el rango de los positivos. A continuación se presenta las líneas de código que realizan esta fase:

```
long long term_to_remove = (text[i] * p_pow) % mod;
text_hash = (text_hash - term_to_remove + mod) % mod;
```

B. Sumar Valor de entrada

Una vez hemos restado el valor de salida al valor total debemos de incluir al nuevo elemento en la ecuación. Para poder hacer esto primero debemos de recorrer los dígitos a la izquierda. Esto se logra multiplicando el hash restante por la base. Una vez hecho esto sumamos el valor del nuevo elemento. De esta manera hemos movido un elemento a la derecha la ventana/subcadena y contamos con un nuevo valor Hash a comparar con nuestros patrones.

```
text_hash = (text_hash * p + text[i + m]) % mod;
```

C. Comparación de Valores Hash

El algoritmo de Rabin-Karp compara los valores hash de las ventanas calculados por el motor de Rolling Hash con los hash de los patrones. No obstante, este algoritmo añade una capa de seguridad ante la probabilidad de falsos positivos causados por colisiones. El Polynomial Hash tiene el riesgo de generar dos valores hash idénticos cuando los elementos de cada subcadena son de hecho diferentes. Cuando el motor encuentra una coincidencia de valores hash, Rabin-Karp hacen una segunda verificación **comparando elemento por elemento directamente de las subcadenas**. Esto solo se realiza si la prueba del hash sale positiva pero, aunque resuelve el problema de las colisiones, ante cadenas con alta probabilidad de colisiones Rabin-Karp baja su eficiencia casi al mismo nivel que Greedy ($O(n \cdot m)$). Es debido a esto que la eficiencia de Rabin-Karp es directamente proporcional a la baja probabilidad de colisiones.

```
if (text_hash == pattern_hash) {  
    // Confirmación carácter por carácter para evitar falsos positivos  
    // por colisión  
        if (text.substr(i, m) == pattern) {  
            return i + 1; // Retornamos la posición basada en 1  
        }  
    }
```

1.3 Complejidad Asintótica

Las variables que definen la complejidad asintótica de esta implementación son las siguientes:

N : Longitud del texto de transmisión.

M : Longitud del patrón o código malicioso.

K : Cantidad de patrones a buscar.

Para esta implementación de Rabin-Karp de patrón único implica los siguientes procesos:

- Precálculo de potencia y primer hash: Toma $O(M)$ operaciones.
- Ventana deslizante (Rolling Hash): Recorre el texto de tamaño N . Cada actualización del hash toma $O(1)$.
- Verificación de coincidencia (substr): En el caso promedio ocurre raramente (solo ante colisión o match real), tomando $O(M)$.
- Complejidad por patrón: $O(N + M)$ en el caso promedio.

Debido a que ejecutamos esta secuencia **por patrón a buscar**, la complejidad total de la solución propuesta es la siguiente:

$$O(K \cdot (N + M))$$

El algoritmo original de Rabin-Karp incluye la característica de **búsqueda multipatrón**. Esto quiere decir que este algoritmo busca en un solo parseo la coincidencia de múltiples patrones al mismo tiempo. Para lograr esto implica tener una estructura de datos que contenga los valores hash de los diferentes patrones agrupados por tamaño. Una vez se hace la comparación, se busca si coincide con alguno de los valores de los patrones guardados. De esta manera comparas múltiples patrones del mismo tamaño a la vez. Mejorando el rendimiento a : $O(N + K \cdot M)$.

Parte 2: Palíndromos

1. Introducción

El programa analiza 4 archivos: transmission01, transmission02, transmission62 y transmission21. Dentro de estos archivos de texto se encuentran cadenas de textos, de los cuales debemos encontrar el palíndromo más largo. Al final la función deberá regresar un par de enteros con el índice donde inicia y termina el palíndromo. Para poder entender un poco la problemática, debemos de entender qué es un palíndromo. Un palíndromo es una palabra que se lee igual de izquierda a derecha y viceversa. Hay diferentes maneras de como realizarlo, pero al final se escogió la técnica Manacher, debido a que es menos costosa en complejidad:

- $O(n^3)$: Tomar la letra inicial y la longitud de la palabra, verificando si realmente es un palíndromo
- $O(n^2)$: Fijar el centro de una palabra y expandir ambos lados, aumentando el contador cada vez que coincidan dos letras

2. Propuesta de soluciones

Para poder programar la solución, primero debemos de comprender cómo funciona el método. Para poder tener una mejor visualización se tomará como ejemplo una cadena utilizada en la implementación de la transmisión 12 (mcode2mcode3edocm). A continuación veremos cómo funciona paso a paso:

1. Paso 1: Creamos una nueva cadena, agregando un carácter especial entre cada letra del string. Además de uno al inicio y al final del string. En este caso estaremos agregando el carácter "|", ya que es el menos probable que aparezca.

string	m	c	o	d	e	2	m	c	o	d	e	3	e	d	o	c	m
posición original	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

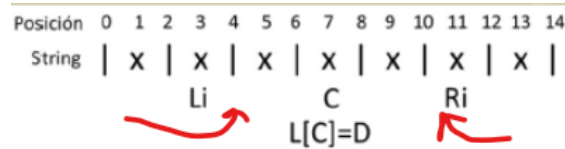
Después de agregarle el caracter, quedaría así:

		m		c		o		d		e		2		m		c		o		d		e		3		e		d		o		c		m	
L	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

Teniendo esta formación siempre tendremos una formación impar de nuestro texto y podremos centrar nuestro string en donde debe de ser.

2. Paso 2: Es tiempo de definir nuestras variables

Tomando en cuenta el centro de nuestro string debemos de identificar el lado derecho y el lado izquierdo, el cual nos va estar ayudando a poder identificar si se encontró algún palíndromo.



C: La última posición calculada de L. Todos los valores en posiciones

- menores o iguales en C en L se conocen.
- Ri : Posición a la derecha de C a la cual se le quiere calcular su valor L.
- Li : Posición a la izquierda de C que está separada exactamente el mismo número de caracteres de C que Ri . Esto implica que $Ri - C = C - Li$.

3. La propiedad de la simetría

De acuerdo con Manacher, si ya se sabe que hay un palíndromo centrado en C, entonces todo lo que esta a la izquierda de la misma es un reflejo exacto de la derecha. Si quieres saber el radio de una posición i que esta en el palíndromo grande no necesito comparar todos los caracteres de 0. Esta formula se utiliza para poder ver los índices que se encuentran dentro del rango.

$$P[i] = \min(R - i, P[i_mirror])$$

4. Guardar los índices

Ya que tienes las posiciones en las que se encuentra el palíndromo deberás de restarle uno a la posición para tener la posición original o lo puedes dejar así. Solo escribiendo que esta posición es con respecto a la mezcla del caracter extra y el string.

3. Complejidad espacial y temporal

Subcadena común más larga entre transmisiones

1. Introducción y planteamiento

El programa analiza dos archivos, `transmission01.txt` y `transmission02.txt`, que representan datos enviados de un dispositivo a otro. Su contenido son solo caracteres hexadecimales (0–9, A–F) y saltos de línea.

La medida elegida es la subcadena común más larga: el bloque de caracteres consecutivos más grande que aparece idéntico en ambos archivos.

La salida esperada es un par de enteros con la posición inicial y la posición final, contando desde 1, de esa subcadena dentro del primer archivo.

Hay que distinguirla de la subsecuencia común más larga. En una subsecuencia los caracteres pueden estar separados. En una subcadena tienen que ser contiguos. Esta diferencia define todo el algoritmo, porque una coincidencia se rompe en cuanto aparece un carácter distinto.

2. Propuesta de solución

La solución usa **programación dinámica**. La idea central es calcular, para cada par de posiciones, cuántos caracteres iguales seguidos terminan justo en ese par.

Relación de recurrencia

Sea $dp[i][j]$ la longitud de la subcadena común más larga que termina en $t1[i-1]$ y en $t2[j-1]$:

```
dp[i][j] = dp[i-1][j-1] + 1 si t1[i-1] == t2[j-1]
dp[i][j] = 0 si t1[i-1] != t2[j-1]
dp[0][j] = dp[i][0] = 0 (caso base)
```

Cuando los caracteres coinciden, la racha de la diagonal superior izquierda se alarga en uno. Cuando no coinciden, la racha se reinicia en 0. La respuesta es el valor más grande de toda la tabla, junto con la fila i donde aparece.

Pasos del algoritmo

1. **Limpieza.** `cleanString` elimina saltos de línea (`\n`, `\r`) y espacios, y deja solo caracteres hexadecimales. Así una subcadena puede continuar aunque el archivo tenga un salto de línea a la mitad.
2. **Dos filas en lugar de una matriz.** Cada celda solo depende de la fila anterior. Por eso basta con guardar dos vectores, `prev` y `cur`, de tamaño $m + 1$, en vez de la matriz completa de $(n + 1) \times (m + 1)$.
3. **Llenado.** Se recorre cada i de $t1$ y cada j de $t2$ y se aplica la recurrencia. Cada vez que `cur[j]` supera el máximo, se guardan `maxLen` y `endPos = i`.

4. **Intercambio.** Al terminar una fila, `swap(prev, cur)` convierte la fila actual en la anterior sin copiar datos.
5. **Reconstrucción.** La posición final es `endPos` y la inicial es `endPos - maxLen + 1`. Como se usa `>` estricto, en caso de empate se reporta la primera aparición en `t1`.

Ejemplo con fragmentos reales

Tomemos `0ebbbe` de `transmission01` y `5ebbcd` de `transmission02`. Las celdas resaltadas forman la diagonal que produce la racha de longitud 3 (`ebb`):

	5	e	b	b	c	d
0	0	0	0	0	0	0
e	0	1	0	0	0	0
b	0	0	2	1	0	0
b	0	0	1	3	0	0
b	0	0	1	2	0	0
e	0	1	0	0	0	0

Tabla 1. Filas: caracteres de t1. Columnas: caracteres de t2. El máximo (3) queda en la fila de la segunda b, que es la posición final en t1.

Código

```
// Solo guardamos dos filas de la matriz dp: la anterior (prev) y la actual (cur)
// cur[j] = longitud del substring comun que TERMINA en t1[i-1] y t2[j-1]
vector<int> prev(m + 1, 0), cur(m + 1, 0);
int maxLen = 0; // longitud del substring comun mas largo encontrado
int endPos = 0; // posicion (base 1) en t1 donde termina ese substring

// Llenado de la tabla fila por fila
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (t1[i - 1] == t2[j - 1]) {
            // Coinciden: extendemos la racha de la diagonal (arriba-izquierda)
            cur[j] = prev[j - 1] + 1;
            // Actualizar MaxLen y endPos si encontramos un subtring comun mas largo
            if (cur[j] > maxLen) {
                maxLen = cur[j];
                endPos = i;
            }
        } else {
            cur[j] = 0; // No coinciden: la racha se rompe
        }
    }
    // La fila actual pasa a ser la anterior para la siguiente iteracion
    swap(prev, cur);
}

if (maxLen == 0) {
    return "No se encontro subcadena comun\n";
}

// El inicio se deduce restando la longitud al final
int startPos = endPos - maxLen + 1;
```

Resultado con los archivos del proyecto

Posición inicial	Posición final	Subcadena	Longitud
24	26	ebb	3

Después de la limpieza, transmission01 tiene 168 caracteres y transmission02 tiene 84 (87 bytes menos 3 saltos de línea). La subcadena ebb aparece en t1 en las posiciones 24 a 26 (...c0**ebb**be8b...) y en t2 dentro de ...35**ebb**cd....

3. Complejidad

Sean n y m las longitudes de t1 y t2 después de limpiarlas.

Etapas	Tiempo	Espacio
Limpieza de ambas cadenas	$O(n + m)$	$O(n + m)$
Llenado de la tabla dp	$O(n \cdot m)$	$O(m)$
Reconstrucción de posiciones	$O(1)$	$O(1)$
Total	$O(n \cdot m)$	$O(n + m)$

Complejidad temporal: $O(n \cdot m)$

Los dos ciclos anidados visitan cada par (i, j) exactamente una vez. En cada visita se hace una comparación, una suma y quizá una actualización del máximo, todo en tiempo constante. swap intercambia los punteros internos de los vectores, así que también es $O(1)$. Con los archivos del proyecto son $168 \times 84 = 14,112$ celdas.

Complejidad espacial: $O(m)$ para la tabla

La matriz completa ocuparía $O(n \cdot m)$ memoria, pero aquí solo se guardan dos filas de $m + 1$ enteros, o sea $O(m)$. Si se agregan las copias limpias de las cadenas, el total es $O(n + m)$. En el caso del proyecto, la tabla usa 2×85 enteros en lugar de 169×85 .

Repositorio

Link a repositorio en Github: https://github.com/Citybanamex16/transmision_de_datos.git

Conclusiones

La implementación de los algoritmos de Rabin-Karp, Manacher y programación dinámica demostró ser una estrategia altamente eficiente para el procesamiento masivo de transmisiones. Al reducir significativamente la complejidad temporal y optimizar el uso de memoria respecto a enfoques ingenuos, estas técnicas garantizan una detección rápida y confiable de patrones maliciosos, estructuras simétricas y coincidencias compartidas en escenarios de alta exigencia operacional.

Referencias

<https://www.geeksforgeeks.org/dsa/string-hashing-using-polynomial-rolling-hash-function/>

- Hernandez, P. P. G. (s. f.). *Subsucesión común máxima (LCS – Longest Common Subsequence)*. <https://pier.guillen.com.mx/algorithms/08-dinamica/08.4-lcs.htm>
- *Rolling Hash: A Powerful Technique for String Manipulation in Programming – AlgoCademy Blog.* (s. f.). https://algotcademy-com.translate.goog/blog/rolling-hash-a-powerful-technique-for-string-manipulation-in-programming/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge
-